West Visayas State University
COLLEGE OF INFORMATION AND COMMUNICATIONS TECHNOLOGY
Luna St., La Paz, Iloilo City 5000
Iloilo, Philippines
* Trunkline: (063) (033) 320-0870 loc 1403 * Telefax No.: (033) 320-0879
* Website: www.wvsu.edu.ph * Email Address: cict@wvsu.edu.ph

# Exercise for Unit 5

Name: Ramuel Carl Q. Barlas Date: _____

Name: _____

Year and Section: BSCS 3A AI

*Note: Save your Python source codes into a single .ipynb file OR multiple python files, representing each number, with the proper naming convention (see Readme on the repository), upload it to your assigned folder in the GitHub organization.*

**Training your own Skipgram.** Train a word embedding model using the skip-gram architecture with negative sampling, then evaluate whether the learned vectors capture useful semantic relationships. Download the source code here and perform the following steps:

1. (10 points) Use a Wikipedia articles as your dataset:
    a. Select a Wikipedia article of your choice as the corpora for this exercise.
    b. The article should be reasonably long (at least a few thousand words) for good results.

c. Determine which part of the code (codeline) this part is:

Answer:

```
# Paste the Wikipedia link you want to use here.
# The article should be reasonably long (at least a few thousand words) for good results.
WIKI_URL = "https://en.wikipedia.org/wiki/Wikipedia"
```

```
def fetch_wikipedia_article(url: str) -> str:
    headers = {
        "User-Agent": "Mozilla/5.0 (compatible; SGNS-Wikipedia-Training/1.0)"
    }
    resp = requests.get(url, headers=headers, timeout=30)
    resp.raise_for_status()

    # Extract main content text from the Wikipedia page
    soup = BeautifulSoup(resp.text, "html.parser")

    content_div = soup.find("div", {"id": "mw-content-text"})
    if content_div is None:
        raise ValueError("Could not find Wikipedia article content.")

    paragraphs = content_div.find_all(["p", "li"])
    text_blocks = []

    for p in paragraphs:
        txt = p.get_text(" ", strip=True)
        if txt:
            text_blocks.append(txt)

    text = "\n".join(text_blocks)

    text = re.sub(r"\[[0-9]+\]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text
```

2. (10 points) Preprocess the text coming from the selected corpus.

a. Determine which part of the code (codeline) this part is:

Answer:

```
def preprocess_text(text: str) -> List[List[str]]:
    sentences = sent_tokenize(text)

    processed = []
    for sent in sentences:
        sent = sent.lower()
        sent = re.sub(r"[^a-z0-9\-\s]", " ", sent)
        sent = re.sub(r"\s+", " ", sent).strip()
        if not sent:
            continue

        tokens = word_tokenize(sent)

        cleaned = []
        for tok in tokens:
            tok = tok.strip("-")
            if not tok:
                continue
            if tok.isdigit():
                continue
            if len(tok) < 2:
                continue
            cleaned.append(tok)

        if len(cleaned) >= 3:
            processed.append(cleaned)

    return processed
```

3. (10 points) Train a Skip-gram with Negative Sampling model.
   a. Execute the code to train a skipgram.
   b. Take note of the following properties with Word2Vec, vector size and window.

c. Determine which part of the code (codeline) this part is:

Answer:

```
## Train SGNS Model
def train_sgns(sentences: List[List[str]]) -> Word2Vec:
    model = Word2Vec(
        sentences=sentences,
        vector_size=100, # What happens if we change this? Try 50, 200, 300 and see how it affects results.
        window=5,
        min_count=1,
        workers=4,
        sg=1,          # 0 = CBOW, 1 = skip-gram
        negative=10,   # negative sampling
        epochs=200,
        sample=1e-3,
        alpha=0.025,
        min_alpha=0.0007,
        seed=RANDOM_SEED,
    )
    return model
```

4. (10 points) Evaluate the embeddings using a small test set.
   a. Evaluate the embedding output using a small test set.

### Relatedness Test

All 8 pairs are covered. The model is able to capture some known semantic relationship such as “wikimedia - foundation” (0.6346, slightly lower than the expected 0.75), and “reference - work” (0.3587, slightly lower than the expected value of 0.45). Others show lowered similarity such as “free - online” (0.2717, lower than the expected 0.55), and “english - wikipedia” (0.3611, lower than the expected 0.70). Pairs such as “misinformation - source” and “censor - information” appropriately showed lows scores (0.1831 and 0.0350 as compared to expected values, 0.10 and 0.05) due to its obscurity and rarity within the article.

### Analogy Test

All 3 analogies are covered but the top-5 predictions show 0% accuracy showing that the predictions are wildly different from the expected answers.

### Direct Similarity Checks

“wikimedia” <-> “foundation”, “online” <-> “encyclopedia”, and “reference” <-> “work“ showed strong similarity (0.6346, 0.3611, and 0.3587 respectively), “english” <-> “wikipedia” showed moderate similarity (0.2657), “censor” <-> “information” showed low similarity (0.0350)

b. Change the list of words to be used for evaluation.
c. Determine which part of the code (codeline) this part is:

Relatedness Test

```
def print_top_neighbors(model: Word2Vec, words: List[str], topn: int = 8):
    print("\n=== Nearest Neighbors ===")
    for word in words:
        if has_word(model, word):
            neighbors = model.wv.most_similar(word, topn=topn)
            print(f"\n{word}:")
            for neigh, score in neighbors:
                print(f"  {neigh:20s} {score:.4f}")
        else:
            print(f"\n{word}: [OOV]")
```

Analogy Test

```
def evaluate_analogies(model: Word2Vec, analogies: List[Tuple[str, str, str, str]]):
    """
    Analogy format: a:b :: c:d
    Checks whether most_similar(positive=[b,c], negative=[a]) returns d.
    """
    covered = 0
    correct = 0
    details = []

    for a, b, c, d in analogies:
        if all(has_word(model, w) for w in [a, b, c, d]):
            covered += 1
            try:
                preds = model.wv.most_similar(positive=[b, c], negative=[a], topn=5)
                predicted_words = [w for w, _ in preds]
                hit = d in predicted_words
                correct += int(hit)
                details.append({
                    "analogy": f"{a}:{b}::{c}:?",
                    "expected": d,
                    "predictions": predicted_words,
                    "correct_in_top5": hit
                })
            except KeyError:
                pass

    accuracy = correct / covered if covered else float("nan")
    return {
        "coverage": covered,
        "total": len(analogies),
        "accuracy_top5": accuracy,
        "details": details
    }
```

Direct Similarity Check

```python
def has_word(model: Word2Vec, word: str) -> bool:
    return word in model.wv.key_to_index

def cosine(model: Word2Vec, w1: str, w2: str) -> float:
    v1 = model.wv[w1].reshape(1, -1)
    v2 = model.wv[w2].reshape(1, -1)
    return float(cosine_similarity(v1, v2)[0][0])

def evaluate_relatedness(model: Word2Vec, test_pairs: List[Tuple[str, str, float]]):
    gold = []
    pred = []
    covered = []

    for w1, w2, score in test_pairs:
        if has_word(model, w1) and has_word(model, w2):
            sim = cosine(model, w1, w2)
            gold.append(score)
            pred.append(sim)
            covered.append((w1, w2, score, sim))

    return {
        "covered_items": covered,
        "coverage": len(covered),
        "total": len(test_pairs),
```

List of Words for Evaluation

```python
## Neighbor Test
probe_words = [
    "wikipedia", "encyclopedia", "wikimedia", "online", "articles",
    "foundation", "article", "free", "knowledge", "wales"
]
print_top_neighbors(model, probe_words, topn=8)

## Relatedness Test
relatedness_test = [
    # These scores in the pairs are just examples, you are the ones providing it.
    # You can adjust them based on your understanding of the article and what you expect the model to learn.
    ("wikimedia", "foundation", 0.75),
    ("online", "encyclopedia", 0.70),
    ("english", "wikipedia", 0.45),
    ("reference", "work", 0.45),
    ("free", "online", 0.55),

    ("systematic","bias",0.25),
    ("misinformation", "source", 0.10),
    ("censor", "information", 0.05),
]
```

```python
## Analogy Test
analogy_test = [
    ("source", "information", "articles", "reference"),
    ("foundation", "organization", "collaboration", "community"),
    ("traffic", "visit", "used", "cited"),
]

analogy_results = evaluate_analogies(model, analogy_test)

print("\n=== Analogy Test Set ===")
print(f"Coverage: {analogy_results['coverage']}/{analogy_results['total']}")
print(f"Top-5 accuracy: {analogy_results['accuracy_top5']}")
for item in analogy_results["details"]:
    print(json.dumps(item, ensure_ascii=False))

# Direct Similarity Checks
print("\n=== Direct Similarity Checks ===")
# Change these pairs based on what you expect to be related/unrelated in the article and what words are in the model.
check_pairs = [
    ("wikimedia", "foundation"),
    ("online", "encyclopedia"),
    ("english", "wikipedia"),
    ("reference", "work"),
    ("censor","information"),
```

West Visayas State University
COLLEGE OF INFORMATION AND COMMUNICATIONS TECHNOLOGY
Luna St., La Paz, Iloilo City 5000
Iloilo, Philippines
* Trunkline: (063) (033) 320-0870 loc 1403 * Telefax No.: (033) 320-0879
* Website: www.wvsu.edu.ph * Email Address: cict@wvsu.edu.ph

5. (10 points) Report nearest neighbors, similarity scores, and test-set performance.
    a. Determine which part of the code (codeline) this part is:

Answer:

```
## Neighbor Test
probe_words = [
    "wikipedia", "encyclopedia", "wikimedia", "online", "articles",
    "foundation", "article", "free", "knowledge", "wales"
]
print_top_neighbors(model, probe_words, topn=8)

## Relatedness Test
relatedness_test = [
    # These scores in the pairs are just examples, you are the ones providing it.
    # You can adjust them based on your understanding of the article and what you expect the model to learn.
    ("wikimedia", "foundation", 0.75),
    ("online", "encyclopedia", 0.70),
    ("english", "wikipedia", 0.45),
    ("reference", "work", 0.45),
    ("free", "online", 0.55),

    ("systematic","bias",0.25),
    ("misinformation", "source", 0.10),
    ("censor", "information", 0.05),
]

rel_results = evaluate_relatedness(model, relatedness_test)

print("\n=== Relatedness Test Set ===")
print(f"Coverage: {rel_results['coverage']}/{rel_results['total']}")
for w1, w2, gold, pred in rel_results["covered_items"]:
    print(f"{w1:10s} - {w2:10s} | gold={gold:.2f} pred={pred:.4f}")

## Analogy Test
analogy_test = [
    ("source", "information", "articles", "reference"),
    ("foundation", "organization", "collaboration", "community"),
    ("traffic", "visit", "used", "cited"),
]

analogy_results = evaluate_analogies(model, analogy_test)
print("\n=== Analogy Test Set ===")
print(f"Coverage: {analogy_results['coverage']}/{analogy_results['total']}")
print(f"Top-5 accuracy: {analogy_results['accuracy_top5']}")
for item in analogy_results["details"]:
    print(json.dumps(item, ensure_ascii=False))

# Direct Similarity Checks
print("\n=== Direct Similarity Checks ===")
# Change these pairs based on what you expect to be related/unrelated in the article and what words are in the mode
check_pairs = [
    ("wikimedia", "foundation"),
    ("online", "encyclopedia"),
    ("english", "wikipedia"),
    ("reference", "work"),
    ("censor","information"),
]
for w1, w2 in check_pairs:
    if has_word(model, w1) and has_word(model, w2):
        print(f"{w1:10s} <-> {w2:10s}: {cosine(model, w1, w2):.4f}")
    else:
        print(f"{w1:10s} <-> {w2:10s}: OOV")
```

=== Nearest Neighbors ===

**wikipedia:**

| | |
|---|---|
| would | 0.3507 |
| registered | 0.3430 |
| alternative | 0.3398 |

etc.

**encyclopedia:**

| | |
|---|---|
| scholarpedia | 0.6204 |
| blunders | 0.6109 |
| apart | 0.5931 |

etc.

**wikimedia:**

| | |
|---|---|
| foundation | 0.6346 |
| selects | 0.5690 |
| gericht | 0.5610 |

etc.

**online:**

| | |
|---|---|
| torn | 0.5657 |
| apart | 0.5612 |
| safety | 0.5453 |

etc.

**articles:**

| | |
|---|---|
| george | 0.4947 |
| anarchism | 0.4917 |
| bush | 0.4913 |

etc.

**foundation:**

wikimedia 0.6346

hillsborough 0.6098

disaster 0.6091

etc.

**article:**

shizuoka 0.5449

draft 0.5419

titled 0.5306

etc.

**free:**

provision 0.5484

trumps 0.5461

faith-based 0.5395

etc.

**knowledge:**

ase 0.5600

graphs 0.5450

intends 0.5346

etc.

**wales:**

jimmy 0.7425

ro 0.5811

christine 0.5737

etc.

=== Relatedness Test Set ===

Coverage: 8/8

wikimedia - foundation | gold=0.75 pred=0.6346

online - encyclopedia | gold=0.70 pred=0.3611

english - wikipedia | gold=0.45 pred=0.2657

reference - work | gold=0.45 pred=0.3587

free - online | gold=0.55 pred=0.2717

systematic - bias | gold=0.25 pred=0.4245

misinformation - source | gold=0.10 pred=0.1831

censor - information | gold=0.05 pred=0.0350

=== Analogy Test Set ===

Coverage: 3/3

Top-5 accuracy: 0.0

{"analogy": "source:information::articles:?", "expected": "reference", "predictions": ["tied", "comprise", "relevant", "living", "biographical"], "correct_in_top5": false}

{"analogy": "foundation:organization::collaboration:?", "expected": "community", "predictions": ["messaging", "anti-abuse", "ceas", "faith", "8th"], "correct_in_top5": false}

{"analogy": "traffic:visit::used:?", "expected": "cited", "predictions": ["nude", "non-sexual", "quakers", "pagerank", "assertions"], "correct_in_top5": false}

=== Direct Similarity Checks ===

wikimedia <-> foundation : 0.6346

online <-> encyclopedia : 0.3611

english <-> Wikipedia : 0.2657

reference <-> work : 0.3587

censor <-> information : 0.0350

West Visayas State University
COLLEGE OF INFORMATION AND COMMUNICATIONS TECHNOLOGY
Luna St., La Paz, Iloilo City 5000
Iloilo, Philippines
* Trunkline: (063) (033) 320-0870 loc 1403 * Telefax No.: (033) 320-0879
* Website: www.wvsu.edu.ph * Email Address: cict@wvsu.edu.ph

Answer the following questions:

1. (10 points) What are the critical parts of the script related on training the Word2Vec?

Answer:

The critical parts of the script in training the Word2Vec is hinged on its parameters and the invocation of the model.

**Model Initialization and Parameters**

The *train_signs* function is what initializes the model used and its given parameters.

```
## Train SGNS Model
def train_sgns(sentences: List[List[str]]) -> Word2Vec:
    model = Word2Vec(
        sentences=sentences,
        vector_size=100, # What happens if we change this? Try 50, 200, 300 and see how it affects results.
        window=5,
        min_count=1,
        workers=4,
        sg=1,          # 0 = CBOW, 1 = skip-gram
        negative=10,   # negative sampling
        epochs=200,
        sample=1e-3,
        alpha=0.025,
        min_alpha=0.0007,
        seed=RANDOM_SEED,
    )
    return model
```

**Model Invocation**

This line of code allows the model to run and train using the given dataset/corpus.

```
model = train_sgns(sentences)
```

(10 points) What Word2Vec methods were used to evaluate the similarity of words?

**most_similar** is used for finding nearest neighbors and solving analogies.

- **model.wv.most_similar(word, topn=topn)** is used for nearest neighbors
- **preds = model.wv.most_similar(positive=[b, c], negative=[a], topn=5)** is used for analogies.

**cosine_similarity** is used to calculate the cosine similarity of words accessed via **model.wv[word]**.

Update the code to:

1. (15 points) Retrain the Word2Vec model with the following configuration:
    a. Window size of 10

    Compare the evaluation values to the previous model. What can you derive from the change?

    === Relatedness Comparison ===
    Original (window=5):
    Coverage: 8/8
    wikimedia - foundation | gold=0.75 pred=0.6346
    online - encyclopedia | gold=0.70 pred=0.3611
    english - wikipedia | gold=0.45 pred=0.2657
    reference - work | gold=0.45 pred=0.3587
    free - online | gold=0.55 pred=0.2717

    New (window=10):
    Coverage: 8/8
    wikimedia - foundation | gold=0.75 pred=0.6914
    online - encyclopedia | gold=0.70 pred=0.4071
    english - wikipedia | gold=0.45 pred=0.2628
    reference - work | gold=0.45 pred=0.3705
    free - online | gold=0.55 pred=0.2803

    === Analogy Test Set ===
    Original (window=5):
    Coverage: 3/3
    Top-5 accuracy: 0.0
    {"analogy": "source:information::articles:?", "expected": "reference", "predictions": ["tied", "comprise", "relevant", "living", "biographical"], "correct_in_top5": false}
    {"analogy": "foundation:organization::collaboration:?", "expected": "community", "predictions": ["messaging", "anti-abuse", "ceas", "faith", "8th"], "correct_in_top5": false}
    {"analogy": "traffic:visit::used:?", "expected": "cited", "predictions": ["nude", "non-sexual", "quakers", "pagerank", "assertions"], "correct_in_top5": false}

    New (window=10):

Coverage: 3/3
Top-5 accuracy: 0.0
{"analogy": "source:information::articles:?", "expected": "reference", "predictions": ["endless", "tied", "acquiring", "draw", "fully"], "correct_in_top5": false}
{"analogy": "foundation:organization::collaboration:?", "expected": "community", "predictions": ["anti-abuse", "ceas", "8th", "messaging", "16th"], "correct_in_top5": false}
{"analogy": "traffic:visit::used:?", "expected": "cited", "predictions": ["quakers", "find", "pagerank", "cheirank", "positioning"], "correct_in_top5": false}

=== Direct Similarity Checks ===
Original (window=5):
wikimedia <-> foundation: 0.6346
online <-> encyclopedia: 0.3611
english <-> wikipedia : 0.2657
reference <-> work : 0.3587
censor <-> information: 0.0350

New (window=10):
wikimedia <-> foundation: 0.6914
online <-> encyclopedia: 0.4071
english <-> wikipedia : 0.2628
reference <-> work : 0.3705
censor <-> information: -0.0003

From the change, it seems that there are minor improvements in Relatedness Test and Direct Similarity Checks, but no changes in Analogy Test. This shows that the changes have made notable minor improvements in identifying the semantic relations between words but it fails to address certain limitations that impacted the inaccuracy of the model.

Further fine-tuning with the model's parameters would be needed to produce notable improvements.

West Visayas State University
COLLEGE OF INFORMATION AND COMMUNICATIONS TECHNOLOGY
Luna St., La Paz, Iloilo City 5000
Iloilo, Philippines
* Trunkline: (063) (033) 320-0870 loc 1403 * Telefax No.: (033) 320-0879
* Website: www.wvsu.edu.ph * Email Address: cict@wvsu.edu.ph

(15 points) Visualize the vectors using PCA. Generate at least 20 known words and feed it on the visualization. Paste the graph below.

PCA: